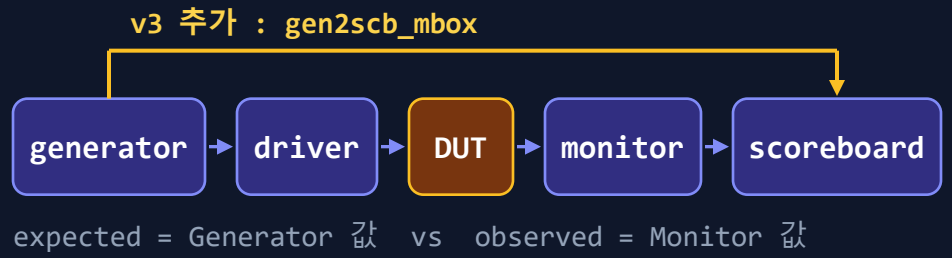
SYSTEMVERILOG · HOMEWORK

RAM TB v2 vs v3

두 검증 모델 차이 비교 분석

tb_ram_v2 · tb_ram_v3 · ram.sv (64 × 8 bit)



expected = Generator 값 vs observed = Monitor 값

정안성

2026. 11

구조

mailbox 2개 → 3개

gen2scb_mbox 추가, Generator 값을 scoreboard로 직접 전달

판정

read만 → 매 cycle

판정 건수 v2 138 / 300 · v3 300 / 300

샘플

posedge → posedge + 1

write 직후 rdata : v2 이전 값 · v3 쓴 값 154 / 154

결과

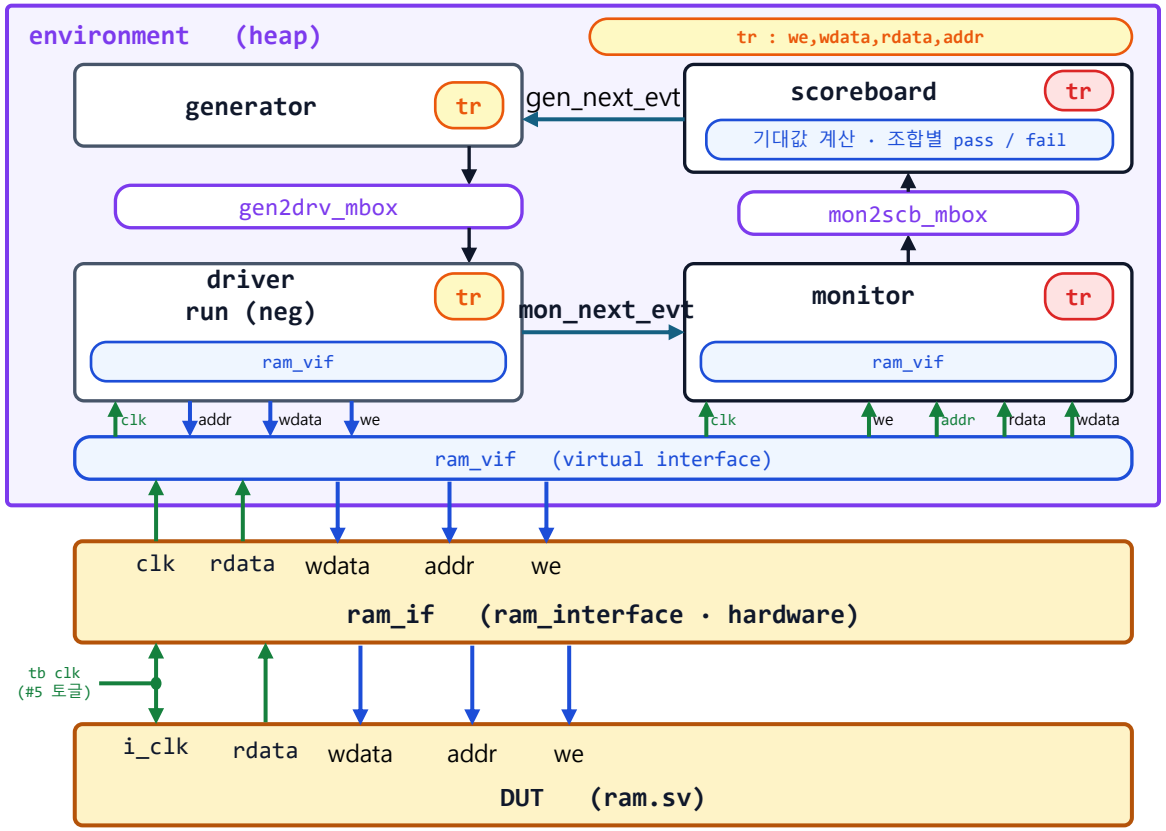
FAIL 0 · 300 tr 완주

v3 write는 rdata로 판정해 DUT까지 확인, addr 경로는 보완 필요

구조: v3는 generator → scoreboard 경로가 추가된 3-mailbox 구조

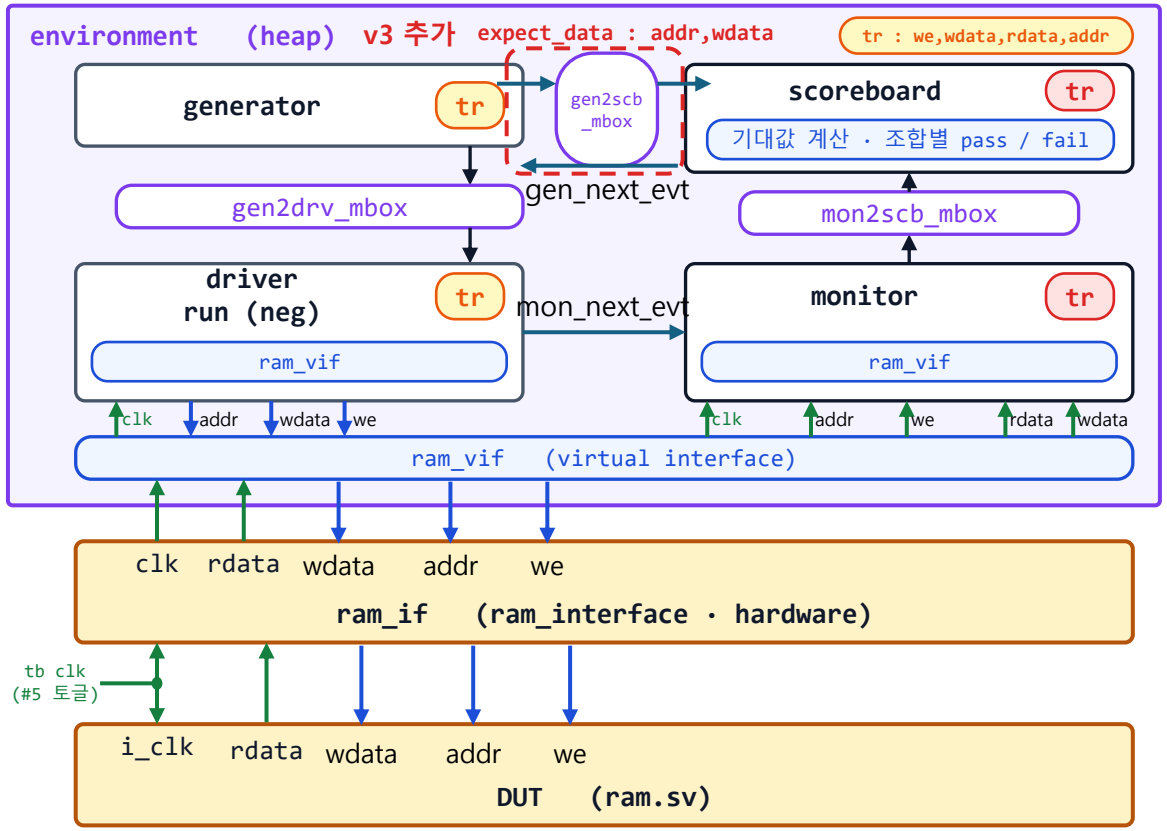
01

결론 · **v2는 Monitor 값만으로 판정, v3는 Generator 값과 DUT 출력(rdata)을 비교**
· **driver · monitor · interface · DUT 구조는 동일 → 차이는 scoreboard로 들어가는 입력 경로**



tb_ram_v2.sv

mailbox 2개 (gen2drv · mon2scb)
scoreboard 입력 : Monitor 관측값 1개 → 기대값도 관측값으로 생성



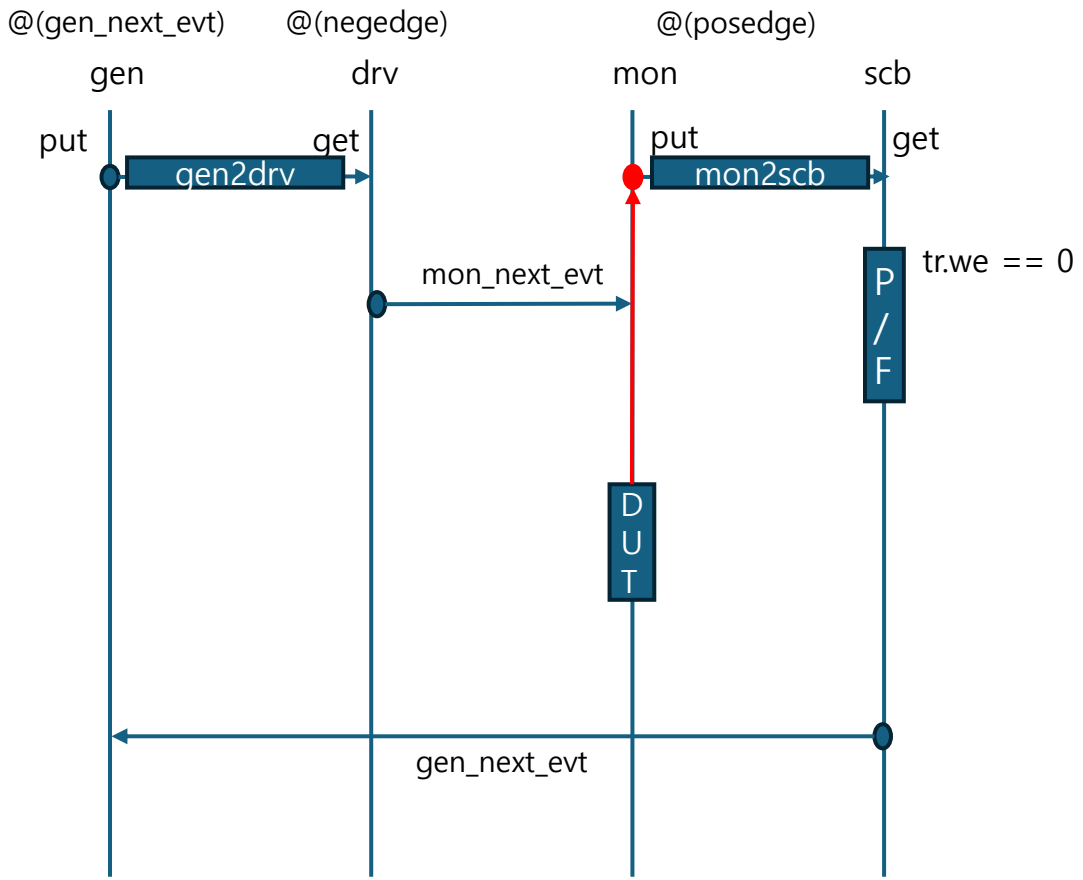
tb_ram_v3.sv

mailbox 3개 (gen2drv · mon2scb · **gen2scb**)
scoreboard 입력 : Generator wdata (기대값) vs Monitor rdata (DUT 출력)

동작 순서: 1 tr = 1 clock 흐름은 같고 판정 입력과 샘플 시점이 다름

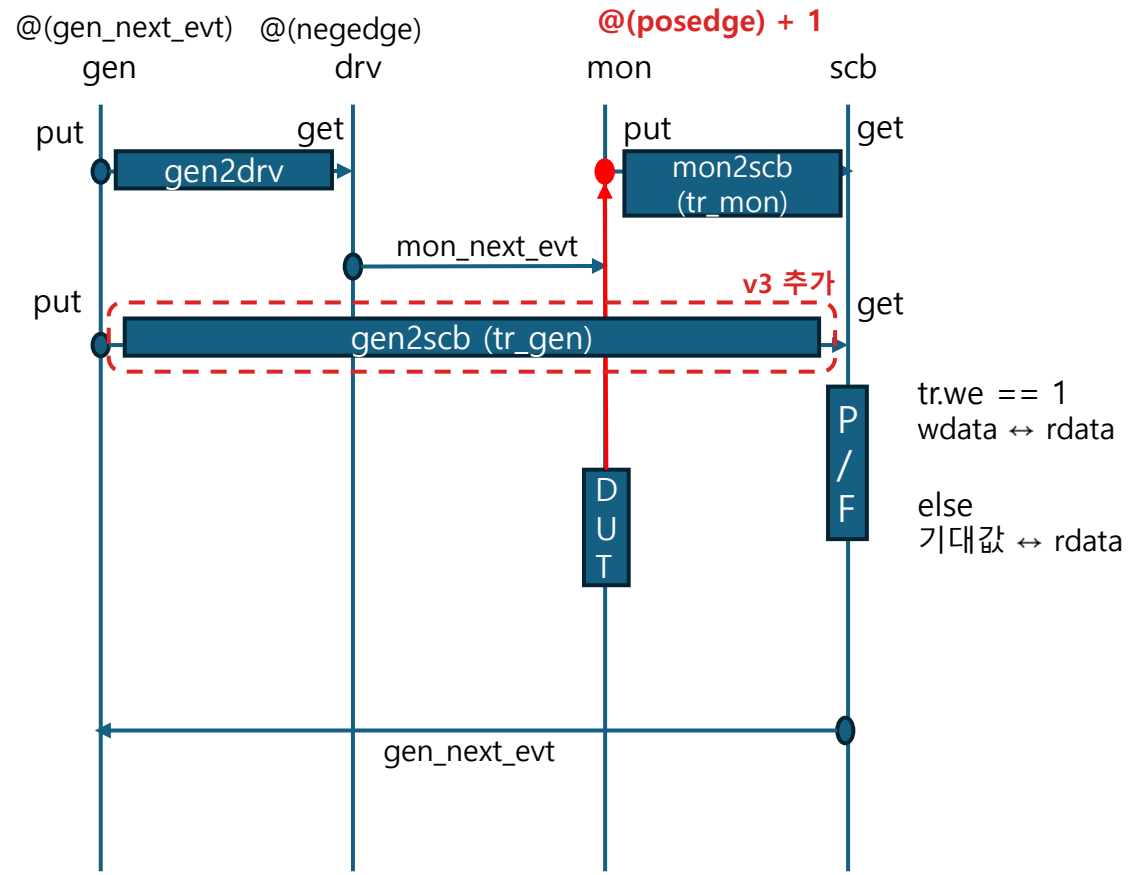
01

결론
- **v3는 generator가 tr을 두 mailbox에 put, scoreboard는 두 값을 다 받은 뒤 판정**
- **monitor 샘플 시점 : v2 @(posedge) → v3 @(posedge) + 1 ns**



`tb_ram_v2.sv`

mon 샘플 → scb get → write면 저장, read면 P/F → gen_next_evt



`tb_ram_v3.sv`

scb는 tr_gen · tr_mon 2개 get 후 → write · read 모두 P/F → gen_next_evt

차이 요약: 수정 지점은 event · 샘플 · 판정 입력 세 가지

01

결론 · **read 판정식은 같고, write 판정 추가로 판정 건수가 138 → 300건으로 증가**
· **v3의 비용 : mailbox 1개 추가, scoreboard가 두 mailbox를 순서대로 기다림**

항목	tb_ram_v2	tb_ram_v3	의미
mailbox	gen2drv · mon2scb (2개)	**+ gen2scb (3개)**	scoreboard가 입력 의도를 직접 받음
기대값 출처	Monitor가 본 wdata	**Generator의 wdata (tr_gen)**	Driver 전달 오류도 비교 대상이 됨
driver → monitor	->> + wait(.triggered)	**-> + @(event)**	둘 다 300건 누락 없음
monitor 샘플	@(posedge)	**@(posedge) + #1**	write 직후 갱신된 rdata 관측 가능
write (we = 1)	기대값 저장만 (판정 없음)	**wdata vs rdata 비교 후 저장**	write 결과를 DUT 출력으로 바로 확인
read (we = 0)	expected === rdata	expected === rdata	동일, DUT 저장값 검증의 핵심
판정 건수	138 / 300	**300 / 300**	v2는 read 건수만큼 판정
report	total + read · write 개수	**read · write별 pass / fail**	어느 동작에서 실패했는지 구분

generator: 만든 tr을 driver와 scoreboard 양쪽에 전달

02

결론 · v3는 gen2scb_mbox를 추가해 Generator 값이 scoreboard로 직접 전달
· randomize · gen_next_evt 대기 흐름은 동일, 전달 경로만 1개 추가

tb_ram_v2.sv · class generator

```
class generator;
    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    event gen_next_evt;

    function new(mailbox #(transaction) gen2drv_mbox,
                 event gen_next_evt);
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox =gen2drv_mbox;
    endfunction

    task run(int run_count);
        repeat(run_count)begin
            tr = new();
            tr.randomize();
            gen2drv_mbox.put(tr);
            tr.debug_print("GEN");
            @(gen_next_evt);
        end
    endtask
endclass
```

tb_ram_v3.sv · class generator

```
class generator;
    transaction tr;
    mailbox #(transaction) gen2drv_mbox;
    mailbox #(transaction) gen2scb_mbox;  // 1
    event gen_next_evt;

    function new(mailbox #(transaction) gen2scb_mbox,  // 1
                 mailbox #(transaction) gen2drv_mbox,
                 event gen_next_evt);
        this.gen2scb_mbox =gen2scb_mbox;  // 1
        this.gen_next_evt =gen_next_evt;
        this.gen2drv_mbox =gen2drv_mbox;
    endfunction

    task run(int run_count);
        repeat(run_count)begin
            tr = new();
            tr.randomize();
            gen2drv_mbox.put(tr);
            gen2scb_mbox.put(tr);  // 2
            tr.debug_print("  GEN  ");
            @(gen_next_evt);
        end
    endtask
endclass
```

1 gen2scb_mbox 추가

선언 · new 인자 · this 대입 3곳에서 mailbox 1개 추가

2 put 2회

gen2drv 직후 같은 시점에 put → scoreboard의 기대값 채널

같은 handle 공유

두 mailbox에 같은 tr 객체가 들어감. driver는 읽기만 하므로 값 충돌은 없음

* 구분선 $display, 미사용 변수 enable은 생략

driver · monitor: 이벤트 방식만 바뀌고 tr 전달 흐름은 동일

02

결론 · v2 ->> + wait(.triggered) → v3 -> + @(event), 두 방식 모두 300건 누락 없음
· @는 트리거 전에 이미 대기 중이어야 함 → v3 로그에서 MON 16 ns 대기 시작, DRV 20 ns 트리거

tb_ram_v2.sv · driver · run()

```
task run();
    forever begin
        @(negedge ram_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("DRV");
        ram_vif.we = tr.we;
        ram_vif.addr = tr.addr;
        ram_vif.wdata = tr.wdata;
        ->>mon_next_evt; // nba tiggered  혼합해서 못사용함 (1)
    end
endtask
```

tb_ram_v3.sv · driver · run()

```
task run();
    forever begin
        @(negedge ram_vif.clk);
        gen2drv_mbox.get(tr);
        tr.debug_print("  DRV  ");
        ram_vif.we = tr.we;
        ram_vif.addr = tr.addr;
        ram_vif.wdata = tr.wdata;
        ->mon_next_evt; // nba tiggered  혼합해서 못사용함 (2)
    end
endtask
```

tb_ram_v2.sv · monitor · run()

```
task run();
    forever begin
        wait(mon_next_evt.triggered);    // blockng assignment (1)
        @(posedge ram_vif.clk);
        tr = new();
        tr.rdata = ram_vif.rdata ;
        tr.we    =    ram_vif.we ;
        tr.addr = ram_vif.addr   ;
        tr.wdata=    ram_vif.wdata ;
        mon2scb_mbox.put(tr);
        tr.debug_print("MON");
    end
endtask
```

tb_ram_v3.sv · monitor · run()

```
task run();
    forever begin
        @(mon_next_evt);    // blockng assignment (2)
        @(posedge ram_vif.clk);
        #1; (3)
        tr = new();
        tr.rdata = ram_vif.rdata ;
        tr.we    =    ram_vif.we ;
        tr.addr = ram_vif.addr   ;
        tr.wdata=    ram_vif.wdata ;
        mon2scb_mbox.put(tr);
        tr.debug_print("  MON  ");
    end
endtask
```

1 ->> + wait(.triggered)

NBA 영역에서 트리거, triggered는 같은 time step 동안 유지 → 경쟁에 강함

2 -> + @(event)

Active 영역에서 즉시 트리거. 대기가 먼저 걸려 있어야 수신

3 #1 추가

샘플 시점을 1 ns 뒤로 이동 → 다음 장

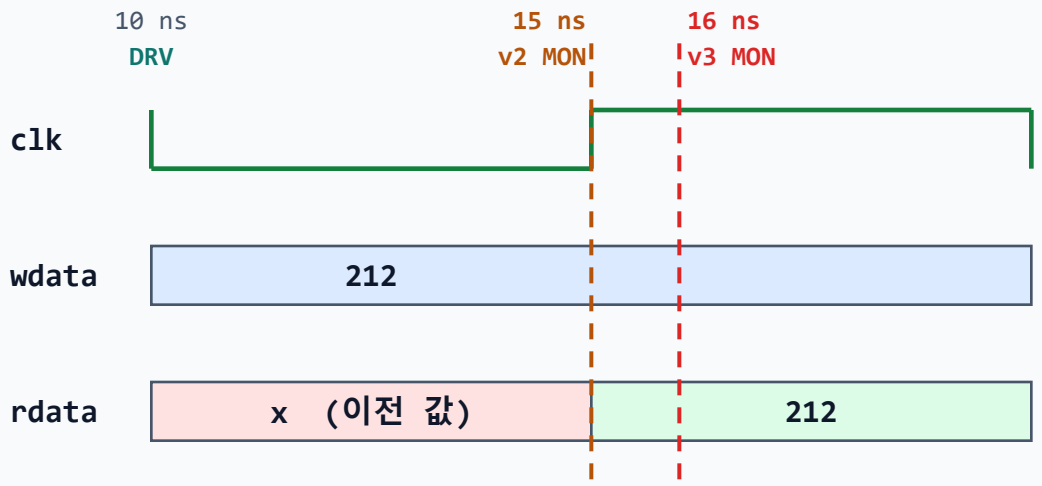
샘플 시점: posedge는 이전 값, +1 ns 뒤에는 쓴 값이 보임

02

결론 · write 샘플에서 rdata = wdata : v2 1 / 162건, v3 154 / 154건

· 원인 : DUT는 ram[addr] <= wdata (NBA) + assign rdata → posedge 순간에는 갱신 전 값

write 1건 타이밍 (첫 tr : addr 16 · wdata 212)



v2 : rdata = x 샘플 v3 : rdata = 212 샘플

xsim log 근거

write 샘플	v2 (posedge)	v3 (posedge + 1)
첫 tr	15 ns · wdata 212 · rdata x	16 ns · wdata 212 · rdata 212
마지막 write	2995 ns · wdata 121 · rdata 193	3006 ns · wdata 20 · rdata 20
rdata = wdata	1 / 162	154 / 154

1 NBA 순서

posedge 순간 monitor(Active)가 먼저 읽고, ram 갱신(NBA)은 그 뒤에 일어남

2 +1 ns 효과

NBA 갱신 + assign rdata 전파 후 샘플 → write 결과를 같은 tr에서 관측 가능

v2 read는 문제없음

read 사이클은 ram 값이 바뀌지 않음 → negedge 구동 후 posedge에서 읽어도 유효 (v2 read fail 0)

scoreboard: write에도 판정을 추가해 매 tr마다 pass / fail

02

결론 · v3는 tr_gen · tr_mon 두 mailbox에서 get → write · read 모두 DUT 출력(rdata)으로 판정
· v2의 write는 expected 저장만 수행, 쓴 값은 이후 read에서 간접 검증

tb_ram_v2.sv · scoreboard · run()

```
task run();
    forever begin
        mon2scb_mbox.get(tr);
        tr.debug_print("SCB");
        total_count ++;

        if(tr.we)begin
            expected_rdata[tr.addr] = tr.wdata;  (1)
            write_count ++;
        end

        else begin
            read_count ++;

            if(expected_rdata[tr.addr] === tr.rdata)begin
                pass_count ++;  (4)
            end

            else begin
                fail_count ++;
                // FAIL $display 3줄 생략
            end
        end

        ->gen_next_evt;
    end
endtask
```

tb_ram_v3.sv · scoreboard · run()

```
task run();
    forever begin
        gen2scb_mbox.get(tr_gen);
        tr_gen.debug_print("SCB-GEN");  (2)
        mon2scb_mbox.get(tr_mon);
        tr_mon.debug_print("SCB-MON");
        total_count ++;

        if(tr_gen.we)begin
            if(tr_gen.wdata == tr_mon.rdata)begin  (3)
                w_pass_count ++;
                t_pass_count ++;
            end
            else begin
                w_fail_count ++;
                t_fail_count ++;
            end
            expected_rdata[tr_gen.addr] = tr_gen.wdata;
            write_count ++;
        end
        else begin
            read_count ++;
            if(expected_rdata[tr_gen.addr] === tr_mon.rdata)begin
                r_pass_count ++;  (4)
                t_pass_count ++;
            end
            else begin
                r_fail_count ++;
                t_fail_count ++;
                // FAIL $display 3줄 생략
            end
        end
        ->gen_next_evt;
    end
endtask
```

1 v2 write
관측한 wdata를 기대값으로 저장, 판정 없음

2 get 2회
tr_gen → tr_mon 순서로 둘 다 도착해야 진행

3 write 판정
Generator wdata와 Monitor rdata를 비교

4 read 판정
=== 로 X까지 비교, 두 버전 동일

03

결과: 두 버전 모두 FAIL 0, 판정 건수는 138 → 300건

결론 · v2 pass 138 = read 건수 / v3 pass 300 = write 154 + read 146
· xsim 재실행으로 두 report 수치 재현 확인 (v2 3005 ns · v3 3006 ns 종료)

tb_ram_v2.sv · report

```
+-------------------------------------------------+
|                  TEST  REPORT                   |
+-------------------------------------------------+
|  total test =   300                             |
|  pass count =   138                             |
|  fail count =     0                             |
+-------------------------------------------------+
| read  count =   138                             |
| write count =   162                             |
+-------------------------------------------------+
```

tb_ram_v3.sv · report

```
+-------------------------------------------------+
|                  TEST  REPORT                   |
+-------------------------------------------------+
|  total test =   300                             |
|  pass count =   300                             |
|  fail count =     0                             |
+-------------------------------------------------+
| read  count =   146                             |
| read  pass  =   146                             |
| read  fail  =     0                             |
+-------------------------------------------------+
| write  count =   154                            |
| write  pass  =   154                            |
| write  fail  =     0                            |
+-------------------------------------------------+
```

항목	v2	v3
total tr	300	300
판정 건수	138	**300**
pass / fail	138 / 0	**300 / 0**
read / write	138 / 162	146 / 154

1 v2 pass 138
read 건수와 동일

2 write 162 미판정
expected 저장만 수행

3 v3 pass 300
전체 tr이 판정 대상

4 write 154 판정
v3에서 새로 집계

03

파형 비교: 같은 write에서 monitor 샘플 시점만 1 ns 차이

결론 · v2는 15 ns, v3는 16 ns에 샘플 → 첫 write의 로그 rdata가 x와 212로 다름
· 파형은 time step 최종값을 표시 → 15 ns 커서에서는 v2도 rdata가 갱신된 값으로 보임

`tb_ram_v2.sv` · 파형

파형 캡처 넣는 자리

`tb_ram_v2` · 0 – 40 ns · 커서 15 ns

`tb_ram_v3.sv` · 파형

파형 캡처 넣는 자리

`tb_ram_v3` · 0 – 40 ns · 커서 16 ns

1 표시 신호

clk · we · addr · wdata · rdata

2 첫 write

addr 16 · wdata 212 (hex d4)
rdata는 15 ns posedge에서 갱신

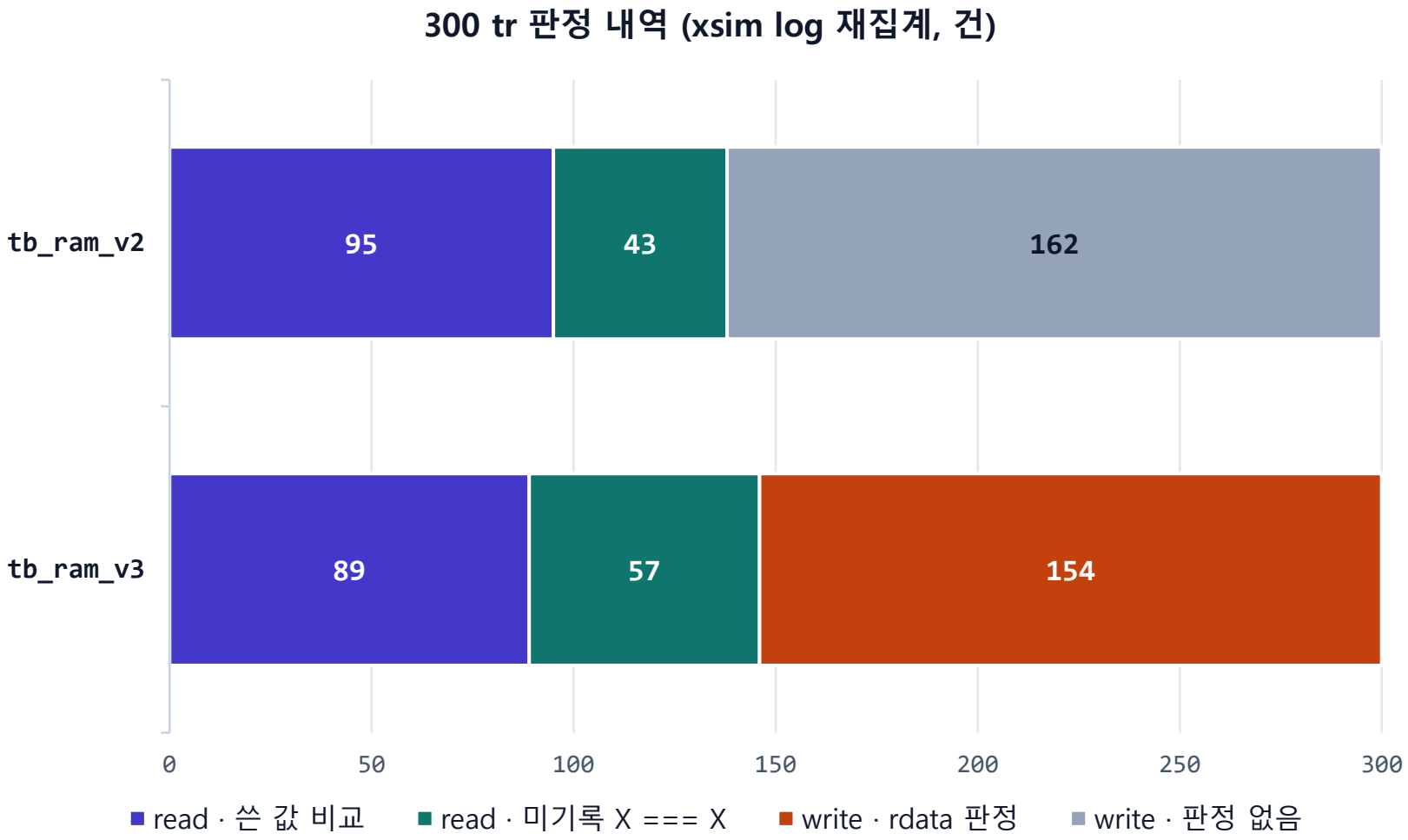
3 샘플 비교

첫 write 파형은 v2 · v3 같음
샘플 차이는 xsim log로 확인

PASS 내역: read의 30 ~ 40%는 아직 안 쓴 주소의 X === X

03

결론 · 실제 저장값 비교는 v2 95건 · v3 89건, 나머지 read는 미기록 주소
· v2 · v3는 stimulus 순서가 달라 (2번째 tr addr : 11 vs 4) 건수 자체의 우열 비교는 불가



쓴 값 비교

v2 95 / 138 (69%)
v3 89 / 146 (61%)
로그 재계산 결과 불일치 0건

X === X

v2 43건 · v3 57건
한 번도 안 쓴 주소를 읽은 경우
logic 배열 + ===로 PASS 처리

write 판정

v3 154건 전부 PASS
쓴 값을 rdata로 바로 확인
고장 검출 결과 → 다음 장

03

분석: v3 write는 DUT까지 검증, Driver addr 경로는 미검증

결론
- **tr_gen.wdata == tr_mon.rdata로 변경 → 쓰기 안 됨 고장을 154 / 154건 해당 cycle에 검출**
- **v2가 놓친 Driver wdata 오류도 검출, Driver addr 오류는 전 버전 미검출 → addr 비교 필요**

판정 범위

판정	비교식	확인 범위	쓰기 안 됨 고장
v2 · v3 read	`expected === rdata`	DUT 저장 · 읽기	95 · 89건 (read)
이전 v3 write	`tr_gen.wdata == tr_mon.wdata`	Driver → interface	**0건**
현재 v3 write	`tr_gen.wdata == tr_mon.rdata`	**DUT 저장 즉시**	**154 / 154건**

두 모델 정리

	v2	v3
장점	구조 단순, mailbox 2개	입력 의도와 관측값 비교
주의	write 결과는 read 때까지 미확인	샘플 타이밍 · 조합 출력 RAM 전제
적합	read 위주 기능 확인	tr 단위로 바로 판정할 때

개선 항목

1 addr · we도 비교

Driver addr 오류 주입 시 전 버전 fail 0
write · read가 같은 틀린 주소를 사용 → gen2scb 값과 비교

```
tr_gen.addr === tr_mon.addr
```

2 미기록 read 줄이기

v3 read의 39%가 X === X
주소 범위를 좁혀 쓴 주소를 다시 읽는 비율을 높임

```
addr inside {[0:15]};
```

3 randc we 유지

매 tr new()로 randc 순환이 끊김
v2 로그 처음 6건 연속 write → 객체 재사용으로 순환 유지

```
tr = new();  // repeat 밖으로 이동
```